

José Carlos López Henestrosa

Ingeniería del software de componentes y sistemas distribuidos

**PEC1 - Arquitecturas de software
distribuido: una solución para cada
problema**

Índice

Ejercicio 1 (20%).....	2
Ejercicio 2 (30%).....	4
Ejercicio 3 (30%).....	6
Ejercicio 4 (20%).....	8
Ejercicio 4.1 (5%).....	8
Ejercicio 4.2 (5%).....	9
Ejercicio 4.3 (5%).....	10
Ejercicio 4.4 (5%).....	11

Ejercicio 1 (20%)

Realizad un estudio comparativo de los estilos arquitectónicos Layered Architecture y Microservices Architecture en función de sus características. Usad la tabla mostrada a continuación.

Por último, describid una decisión arquitectónica específica para cada uno de los dos estilos anteriores que suponga una regla o restricción técnica que el equipo de desarrollo debería seguir para mantener la integridad del estilo elegido.

	Layered architecture	Microservices architecture
Partitioning type	Technical	Domain
Number of quanta	1	1 to many
Deployability	★	★★★★★
Elasticity	★	★★★★★★
Evolutionary	★	★★★★★★
Fault tolerance	★	★★★★★
Modularity	★	★★★★★★
Overall cost	★★★★★★	★
Performance	★★	★★
Reliability	★★★★	★★★★★
Scalability	★	★★★★★★
Simplicity	★★★★★★	★
Testability	★★	★★★★★

En una **arquitectura por capas (Layered)**, una regla técnica fundamental es que solo las capas de negocio y de servicios pueden acceder a la base de datos, lo que impide que la capa de presentación realice llamadas directas a la base de datos. Esta regla ayuda a mantener la separación de responsabilidades, de modo que cada capa se centra en realizar unas tareas concretas sin sobrecarga. Por ejemplo, en una arquitectura por capas, la capa de presentación solo debe gestionar las entradas y salidas del usuario, mientras que la capa de negocio se encarga de la lógica de negocio y la capa de acceso a datos gestiona las interacciones con la base de datos.

Por otro lado, en la **arquitectura de microservicios (Services)**, cada servicio debe ser un componente que se pueda sustituir y actualizar de forma independiente, lo cual facilita la

implementación y permite un acoplamiento débil entre los servicios. Esto permite una mayor flexibilidad y escalabilidad, ya que cada servicio se puede desarrollar, implementar y escalar de forma independiente. El equipo de desarrollo debe tener en cuenta las ventajas y desventajas de los microservicios, como el aumento de la complejidad y los costes operativos, y optar por este modelo solo cuando la aplicación requiera escalabilidad y elasticidad.

Ejercicio 2 (30%)

Photo&Film4You, una tienda de material audiovisual especializada en equipos para rodajes de películas y sesiones profesionales de fotografía, pretende añadir una nueva funcionalidad para recuperar contenido audiovisual generado con tecnologías del siglo XX, implementando un flujo de procesamiento estructurado por etapas consecutivas. El cliente deja físicamente el medio antiguo, en formatos como VHS, Betamax, DVD, CD o cintas de audio, en una oficina de Photo&Film4You. En un primer paso, se añade una pegatina con un código QR que identifica de manera única cada medio, y los discos y cintas se almacenan en armarios diseñados para protegerlos.

A continuación, los operarios retiran los medios de los armarios e introducen cada medio (soporte) en el único equipo de digitalización disponible, donde se procesa de manera ordenada y se genera una salida en un formato estándar de video o audio, que se almacena directamente en un sistema de almacenamiento de objetos (Amazon S3). Cada medio sigue un camino definido de principio a fin, y el equipo indica mediante una señal luminosa cuando ha finalizado el procesamiento de un medio.

En ese momento, el medio se extrae, se verifica mediante el código QR y se guarda en el armario de medios digitalizados, mientras que el siguiente medio inicia su propio procesamiento siguiendo exactamente el mismo recorrido.

En las fases posteriores, un operador revisa y ajusta manualmente el medio digitalizado según sea necesario. Finalmente, una vez completadas todas las transformaciones y revisiones, el video o audio queda disponible en su formato digital final, listo para almacenamiento o uso posterior, y el usuario puede acceder al resultado directamente desde la tienda online de Photo&Film4You, dentro de su área de pedidos.

¿Qué estilo arquitectónico del “Capítulo II: Estilos Arquitectónicos” del libro *Fundamentals of Software Architecture* crees que se adapta mejor a los nuevos requisitos relacionados con la digitalización de medios antiguos, como VHS, Betamax, DVD, CD y cintas de audio?

Justifica tu respuesta comparando los nuevos requisitos y restricciones del proyecto con las ventajas y desventajas del estilo arquitectónico que elijas.

NOTA: En este ejercicio no se debe analizar el estilo arquitectónico del sistema encargado de gestionar el catálogo de material para alquiler, sino únicamente el de este nuevo subsistema.

El flujo actual de Photo&Film4You es convertir medios analógicos en digitales. Si adaptamos este proceso a uno de los estilos arquitectónicos del libro, llegamos a la conclusión de que la arquitectura de *Pipelines* es la que mejor se adecúa, ya que nos permite procesar datos a través de una secuencia de etapas o componentes independientes (filtros) que se comunican a través de canales (*pipelines*). Este patrón es el estándar de la industria para flujos de transformación de datos, lo cual es exactamente la actividad que se busca llevar a cabo.

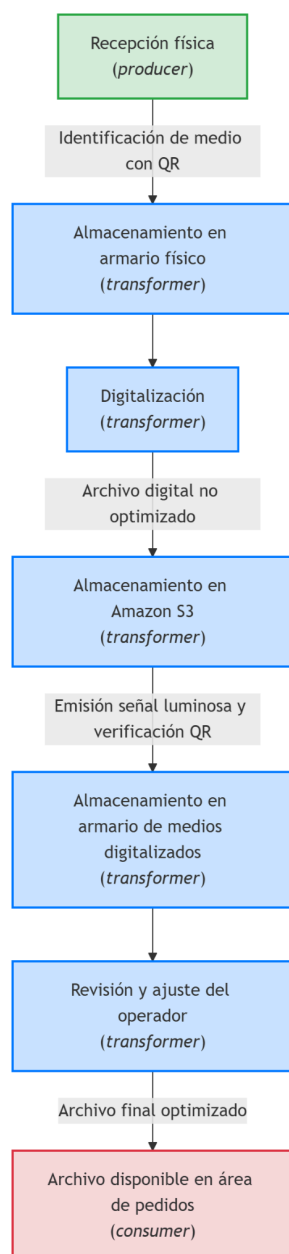


Fig. 1. Flujo de la arquitectura Pipeline

Si nos adentramos con más profundidad en el flujo a desarrollar, podemos representarlo tal y como se muestra en la Fig. 1.

Como podemos apreciar, la implementación de este estilo arquitectónico para este caso deriva en una topología muy sencilla compuesta de procesos secuenciales con un flujo unidireccional. Además, como los filtros son independientes, podemos tener diferentes subprocesos o configuraciones en la máquina de digitalización dependiendo del tipo de medio físico de entrada (VHS, DVD, CD...), ya que la salida hacia el siguiente filtro (*Almacenamiento en armario de medios digitalizados*) siempre será un archivo estandarizado en S3. Si por algún motivo el equipo de desarrollo necesita cambiar el funcionamiento de algún filtro, puede hacerlo sin afectar a la lógica del resto.

No obstante, este estilo también presenta algunas desventajas. Según Richards y Ford, en una arquitectura *Pipeline* el rendimiento general de todo el sistema está estrictamente limitado por el filtro más lento. En este subsistema, identificamos al filtro *Digitalización* como el principal cuello de botella del flujo, ya que hay un único equipo de digitalización disponible. Por otra parte, este filtro presenta otro problema, ya que el procesamiento se interrumpe si la máquina de digitalización falla. Esto implica que el negocio se detiene en esa etapa. No obstante, al modelarlo como un filtro con almacenamiento intermedio (armarios físicos antes y después de la digitalización, incluyendo Amazon S3), logramos que los procesos no completados no se pierdan. Los medios etiquetados siguen a salvo en el armario y los vídeos ya digitalizados continúan disponibles para la revisión del operador. En este sentido, la operativa no se bloquearía por completo a pesar de un posible fallo en dicho filtro.

Ejercicio 3 (30%)

Se propone, sobre la misma funcionalidad de recuperación de contenidos generados en medios físicos, analizada en el ejercicio 2, un ajuste en los flujos de proceso, con el objetivo de mejorar la productividad. Gracias a una inversión extraordinaria, la oficina de Photo&FilmYou ahora cuenta con 10 equipos de digitalización, lo que permite procesar los medios de forma más eficiente. El cliente dejará físicamente el medio antiguo en los mismos formatos contemplados en el ejercicio anterior, con el fin de digitalizarlo y mejorar su calidad.

Cuando un cliente deja su medio físico, se añade una pegatina con un código QR que lo identifica de forma única. Tras la recepción, los discos y cintas se almacenan en armarios diseñados para protegerlos. El procesado y almacenamiento de los medios digitalizados se mantiene respecto al ejercicio anterior. Aunque en ciertos momentos existe intervención humana, que puede implicar que los flujos queden momentáneamente parados, la incorporación de las 10 máquinas permite paralelizar tareas, de modo que varios medios se procesan simultáneamente. Cada equipo indica mediante una señal luminosa cuando finaliza el procesamiento de un medio; en ese momento, el medio se extrae, se lee el código QR para identificarlo y se guarda en el armario de medios digitalizados, mientras que el siguiente medio se introduce en el equipo disponible, manteniendo un flujo constante y eficiente de digitalización.

Posteriormente, los agentes de IA analizan cada parte del recurso digitalizado para detectar posibles deficiencias de calidad y aplicar mejoras automáticas cuando sea necesario. No obstante, en algunas ocasiones la IA puede generar “alucinaciones”, realizando modificaciones proactivas que no siempre son correctas. Por ello, un operador de calidad participa en las fases finales del proceso, revisando y ajustando manualmente los fotogramas que la IA no haya podido mejorar adecuadamente o que presenten inconsistencias, utilizando en caso necesario la digitalización en crudo generada en las fases iniciales como referencia para garantizar la fidelidad del contenido.

Una herramienta de gestión proporciona de manera visual, sobre el perfil de este operador, un seguimiento de las notificaciones de los distintos medios que va recibiendo, facilitando la priorización y el control del trabajo pendiente. Finalmente, una vez completadas las mejoras automáticas validadas y la intervención del operador de calidad, el vídeo o audio queda disponible en su formato digital final, listo para almacenamiento o uso posterior, y el usuario puede ver el resultado directamente en la tienda online de Photo & Film4You dentro de sus pedidos, accediendo a la versión digital de su contenido.

¿Qué estilo arquitectónico del “Capítulo II: Estilos Arquitectónicos” del libro *Fundamentals of Software Architecture* crees que se adapta mejor a los nuevos requisitos relacionados con la digitalización de medios antiguos, como VHS, Betamax, DVD, CD y cintas de audio?

Justifica tu respuesta comparando los nuevos requisitos y restricciones del proyecto con las ventajas y desventajas del estilo arquitectónico que elijas.

NOTA: En este ejercicio no se debe analizar el estilo arquitectónico del sistema encargado de gestionar el catálogo de material para alquilar, sino únicamente el de este nuevo subsistema.

El escenario que nos propone este ejercicio es mucho más complejo que el del ejercicio 2. Ahora contamos con paralelismo, ya que disponemos de 10 máquinas, y con procesos asíncronos que pueden tardar bastante, como el proceso de análisis de la IA. También pueden producirse errores durante el proceso (alucinaciones), que rompen el flujo secuencial y obligan a consultar el archivo original. Además, hay un sistema de notificaciones para que el operador gestione una cola de trabajo pendiente.

En base al “Capítulo II: Estilos Arquitectónicos” del libro *Fundamentals of Software Architecture*, el estilo que mejor se adapta a estos nuevos requisitos es la arquitectura orientada a eventos (*Event-Driven Architecture*). Ahora, el flujo no es estrictamente lineal ni síncrono, ya que el operador de calidad, además de recibir el archivo del paso anterior, también recibe notificaciones. Dispone de una bandeja de entrada para priorizarlas, las cuales pueden contener alertas que obliguen al operador a ir a un paso anterior para recuperar la digitalización en crudo si la IA alucina. La arquitectura orientada a eventos permite que los componentes estén completamente desacoplados y reaccionen a lo que sucede en el sistema mediante mensajes o eventos.

Otra baza a favor de esta arquitectura para este escenario reside en su escalabilidad, ya que nos permite tener una cola de cintas pendientes. Las 10 máquinas actúan como *competing users*: Cuando una máquina termina, emite un evento y pide el siguiente trabajo, lo cual lo convierte en un sistema eficiente para soportar paralelismo. En este caso, el emisor no espera al receptor, por lo que el receptor puede estar realizando otras tareas mientras que las máquinas de digitalización y la IA siguen trabajando mientras se van acumulando notificaciones en la cola del operador.

Por otro lado, un *event broker* (gestor de eventos) permite enrutar y clasificar los mensajes. El sistema puede etiquetar eventos con prioridad alta y la interfaz del operador simplemente se suscribe a estos eventos para organizarlos visualmente. Al estar desacoplado, el operador no depende de que el filtro anterior le pase todo masticado.

Como desventaja, identificamos la falta de un flujo centralizado. Esto dificulta la localización de cada pedido en caso de fallo, ya que no hay un control centralizado. Además, dada la complejidad del sistema, hay que definir eventos muy concretos para casos muy diversos, ya que pueden producirse errores a raíz de situaciones no contempladas. En este sentido, si falla la generación de un evento, el sistema lógico se desincroniza del físico, lo que puede provocar un verdadero caos. Un ejemplo de ello sería que la IA modificara un archivo en S3 justo cuando el operador intenta abrirlo, lo que podría provocar un error no controlado que podría confundir al operador e incluso corromper el archivo en S3 sin entender el porqué.

Ejercicio 4 (20%)

Ejercicio 4.1 (5%)

La frontera entre las decisiones arquitectónicas y las decisiones de diseño muchas veces son complicadas y la frontera entre una y otra no está definida de forma nítida. Catalogad la decisión “Elegir entre una base de datos NoSQL (MongoDB) o una relacional (PostgreSQL) para el sistema de alquiler” entre decisión arquitectónica o decisión de diseño y argumentalo usando los tres criterios estudiados que os pueden ayudar a determinar si una decisión se sitúa más en el lado de la arquitectura o del diseño.

Esta decisión se trata de una **decisión arquitectónica**. Para justificarlo, recordamos que el libro propugna que la frontera entre arquitectura y diseño se traza evaluando si la decisión define o impacta en lo que Richards y Ford llaman las características arquitectónicas (los *ilities* como escalabilidad, rendimiento, etc.).

El libro establece estos tres criterios cruciales para identificar si un requisito o decisión pertenece al ámbito de la arquitectura:

1. **Especifica una consideración de diseño ajena al dominio (*specifies a non-domain design consideration*)**

La decisión trata sobre las cuestiones operativas (el "cómo" lo soporta), no sobre las reglas de negocio o la funcionalidad en sí (el "qué" hace el sistema). En este caso, elegir entre Mongo o Postgres no cambia el hecho de que la tienda alquile cámaras o reciba devoluciones. Sin embargo, sí define cómo el sistema va a manejar la consistencia de los datos, la disponibilidad y la escalabilidad. Es una decisión puramente técnica y operativa, no de negocio.

2. **Influye en algún aspecto estructural del diseño (*influences some structural aspect of the design*)**

Optar por PostgreSQL (relacional) implica diseñar esquemas rígidos, usar migraciones estructuradas y, probablemente, un ORM en la capa de persistencia. Por el contrario, elegir MongoDB (orientado a documentos) fomenta datos desnormalizados, esquemas flexibles y cambia por completo la forma en que se estructuran las transacciones y las consultas. Es una decisión que dicta las reglas estructurales para los programadores.

3. **Crítica o importante para el éxito de la aplicación (*critical or important to application success*)**

Son decisiones de alto impacto y de difícil reversibilidad. Si el sistema de alquiler crece y a los dos años descubrimos que la base de datos no soporta la carga o no garantiza la consistencia del inventario, estamos ante un problema grave. Migrar de una base de datos relacional a una NoSQL (o viceversa) con el sistema ya en producción es extremadamente costoso, arriesgado y lento. Por tanto, es una decisión crítica para la supervivencia y el éxito del software.

Ejercicio 4.2 (5%)

¿Cuáles de estas tareas son responsabilidades de un arquitecto y cuáles son responsabilidad del equipo de desarrollo?

TAREA	RESPONSABLE	JUSTIFICACIÓN
Negociar con los stakeholders para conseguir la aprobación de una decisión que aumenta los costes operativos a corto plazo, pero mejora la escalabilidad.	Arquitecto de software	El arquitecto actúa como enlace entre la tecnología y el negocio. Analizar <i>trade-offs</i> (como coste vs. escalabilidad) y justificar estas decisiones ante perfiles no técnicos es una de sus responsabilidades principales.
Optimizar la lógica interna y el uso de memoria de un algoritmo de búsqueda dentro de un componente de negocio específico.	Equipo de desarrollo	Esto es un detalle de implementación y diseño a nivel de código. Afecta al interior de un componente, no a la estructura global ni a la comunicación del sistema.
Evaluar la viabilidad de la arquitectura actual frente al impacto de nuevas tendencias tecnológicas, como la integración de servicios de IA generativa.	Arquitecto de software	El arquitecto debe tener visión de futuro y analizar continuamente el entorno tecnológico. Es su trabajo que la arquitectura sea evolutiva y pueda escalar con nuevos requerimientos del mercado.
Codificar un recubrimiento para una librería de terceros con el fin de simplificar su uso por parte de otros programadores del equipo.	Equipo de desarrollo	Aunque el arquitecto pueda recomendar el seguimiento de buenas prácticas de programación, la aplicación práctica de las mismas es responsabilidad del equipo de desarrollo.
Definir las funciones automáticas para monitorizar que el acoplamiento entre servicios no supere los límites establecidos.	Arquitecto de software	Esto hace referencia a crear <i>fitness functions</i> . El arquitecto, aparte de diseñar, tiene que vigilar que el equipo no degrade la arquitectura con el tiempo.
Configurar el formato de las trazas de depuración para un módulo concreto del sistema.	Equipo de desarrollo	El arquitecto puede definir una regla general, como que todo el sistema use un registro centralizado, pero configurar el detalle fino de las trazas para un módulo específico es una tarea puramente operativa del programador de ese módulo.

Ejercicio 4.3 (5%)

Supongamos que diseñamos un sistema de pujas en tiempo real (como una plataforma de subastas masivas online) que sigue una arquitectura *Event Driven*. Poned ejemplos concretos donde se vea que no es posible maximizar simultáneamente todas las características deseables para el sistema.

En una subasta, el objetivo ideal es que el usuario sienta que su puja se registra en milisegundos. Sin embargo, si queremos que matemáticamente su puja sea la más alta en ese instante exacto antes de confirmarla, el sistema tendría que bloquear el estado del artículo, detener otros procesos y validar a nivel global el estado. Esta espera destruiría la baja latencia. Por ello, la arquitectura nos obliga a aceptar la consistencia eventual: el sistema acepta el evento de la puja al instante para ser ultrarrápido, pero la validación real ocurre de forma asíncrona milisegundos después. En este caso, asumimos el riesgo de tener que notificar al usuario a posteriori que alguien pujó antes que él.

Por otro lado, nos encontramos con un conflicto entre la escalabilidad y el orden estricto de los eventos. Si la plataforma sufre un pico enorme de tráfico con cien mil personas pujando por el mismo artículo, la única forma de que una arquitectura orientada a eventos soporte esa carga es paralelizando el trabajo. Para escalar, dividimos el flujo de eventos en múltiples particiones procesadas por decenas de consumidores simultáneos. El problema es que, al procesar en paralelo, perdemos la garantía de que los eventos se procesen en el orden cronológico exacto en el que entraron. Si forzáramos al sistema a mantener un orden estricto para ser absolutamente justos, tendríamos que enrutar todas las pujas a una sola cola procesada por un único consumidor, lo que crea un cuello de botella monumental que colapsaría bajo carga. De nuevo, tendríamos que elegir entre escalar procesando en paralelo o mantener el orden colapsando el sistema.

Ejercicio 4.4 (5%)

Elegid dos expectativas que creáis que están asociadas al rol de arquitecto que se hayan debatido en el debate de la actividad, y poned un ejemplo de tarea concreta por cada una de ellas. Los ejemplos elegidos no pueden aparecer en el libro *Fundamentals of Software Architecture*.

Expectativa 1

“Como bien dicen mis compañeros, es difícil encontrar consenso sobre que es un Arquitecto de Software. Yo lo definiría como la persona encargada de sentar las bases sobre como se va a desarrollar un proyecto y supervisarlos según unos criterios previamente definidos buscando siempre el mejor Trade-off posible, ya que nunca vamos a poder disponer de todas las funcionalidades.

En lo que si que coincide la mayoría de bibliografía es en las funciones que el arquitecto tiene que tener. Como mis compañeros ya han destacado casi todas, yo me voy a centrar en dos que me parecen vitales.

Conocer el negocio para el que se desarrolla el sistema: un arquitecto, a parte de conocer la parte técnica de como montar el proyecto, necesita tener nociones sobre el negocio para el cual se va a implementar el software ya que gracias a esto podrá tomar decisiones técnicas sobre como desarrollarlo atendiendo a las necesidades específicas del sector.

Estar actualizado sobre nuevas tecnologías que puedan suponer una oportunidad: el arquitecto necesita estar continuamente en formación ya que pueden aparecer nuevas tecnologías que supongan una mejora enorme en el proyecto y su función es evaluar si la adopción de esta tecnología merece la pena según el momento del proyecto.

Un saludo!!”

En este caso, la expectativa del rol de arquitecto es conocer el negocio para el que se desarrolla el sistema. Como ejemplo para ilustrar este rol, podemos tratar el caso de la empresa ficticia Salud365, la cual nos pide diseñar el flujo de encriptación y anonimización de sus datos. El arquitecto, el cual conoce las reglas estrictas de negocio y las normativas legales del sector médico, toma la decisión arquitectónica de dictaminar en qué capa exacta del sistema deben enmascarse los historiales clínicos antes de que cualquier evento sea emitido a un servicio externo de métricas o marketing.

Expectativa 2

“Buenos días,

Coincido bastante con lo que se ha ido comentando hasta ahora. Podríamos definirlo como una persona que se va a encargar de diseñar la estructura inicial a seguir para intentar cumplir con los objetivos del proyecto que se está llevando a cabo, y resolver esas situaciones en las que surge algún problema y no se sabe que opción elegir ya que no hay una "solución perfecta".

También creo que algo que no se ha mencionado y es bastante importante es lo que NO hace. Al final, el arquitecto no es quien implementa cada detalle del sistema ni desarrolla todas las funcionalidades (o incluso no tiene la capacidad de hacerlo con la misma efectividad que alguien especializado en ello). Como además su función principal es otra, no debería ser alguien que pueda convertirse en un cuello de botella tomando decisiones técnicas que quizá puedan pertenecer al equipo de desarrollo. Es decir, su visión es más "desde fuera", no debe conocer al detalle cada implementación de la lógica del negocio (aunque pueda ayudar con tareas menos prioritarias para no quedarse oxidado).

Por otro lado, igual que la definición de esta figura ha ido evolucionando a lo largo del tiempo, seguramente seguirá haciéndolo y la imagen que tenemos ahora de arquitecto de software cambiará dentro de unos años, porque seguro que aparecen nuevas herramientas y se definen nuevas arquitecturas que ahora no existen. Esto hace que sea muy importante, tal y como comentan Cristian y Javier, que siga aprendiendo y adquiriendo más "technical-breath", es decir, conociendo que existen más herramientas y maneras de hacer las formas aunque no se especialice en ellas.

Creo que podemos llegar más o menos todos a la misma conclusión, en que es una figura que se encarga de guiar el proyecto y establecer la dirección que tiene que seguir y cómo hacerlo, resolviendo los problemas que puedan surgir en el camino y teniendo en cuenta los compromisos que se tienen al elegir cada opción.

Un saludo.”

En este otro caso, la expectativa del rol de arquitecto es mantener una visión de alto nivel y evitar ser un cuello de botella en la implementación. Un ejemplo propio para esta expectativa sería dentro del contexto de la integración de un sistema de pagos en la aplicación mencionada en la expectativa anterior (Salud365). En este caso, el arquitecto define la topología de red, el protocolo de comunicación y establece el contrato de la API para que los servicios se comuniquen de forma segura. Sin embargo, delega expresamente en el equipo de desarrollo la elección de la librería específica de parsing de JSON a utilizar o la estructura interna de los bucles del algoritmo. De esta forma, los programadores avanzan sin tener que esperar la validación del arquitecto para cada línea de código.